

Семинар №10

Умные указатели

Мотивация

- Хотим более аккуратное использования памяти (и вообще разделяемых ресурсов)
- Можем забыть написать `delete` или вылетит исключение – избегаем утечек памяти с помощью RAII
- Хотим следить за тем, кто владеет ресурсами
- `std::unique_ptr` – уникальное владение ресурсами
- `std::shared_ptr` – разделяемое владение ресурсами

std::shared_ptr

- Можем создать от указателя, на который уже выделена память
- Нельзя создавать два умных указателя от одного обычного указателя, иначе – double free
- При копировании увеличивается счетчик ссылок
- При разрушении объекта счетчик ссылок уменьшается
- Когда счетчик ссылок опускается до нуля, вызывается деструктор объекта

Deleter и Allocator

- Можно установить кастомный делитер – механизм освобождения памяти
- Также можно установить кастомный аллокатор
- Можно использовать для контроля за другими ресурсами (файловые дескрипторы, сокеты и др.), а также для кастомного контроля памяти (например, использовать malloc/calloc и free).

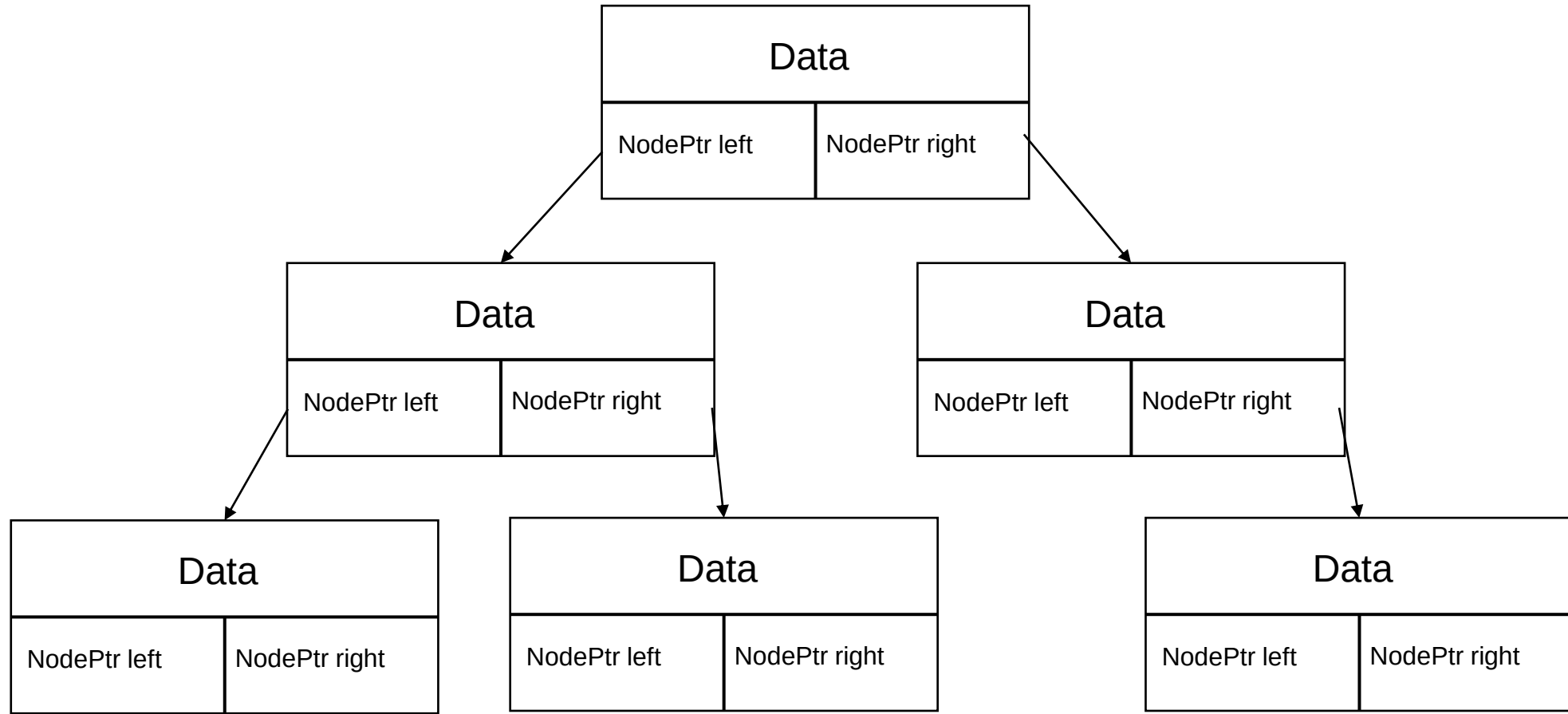
std::make_shared

- `make_shared` принимает аргументы конструктора и сразу создаёт объект, возвращая `shared_ptr`
- Решает проблемы:
 - Нельзя случайно создать два набора умных указателей по одному сишному указателю
 - Избавляемся от необходимости работать с указателями вообще (например, указатель мог быть испорчен)
 - Экономим на выделениях памяти (можем одной аллокацией выделить и `control block` (счетчик), и сам объект)
 - На деаллокациях тоже экономим
 - Лучше кэш-локальность
 - Лучше при работе с исключениями

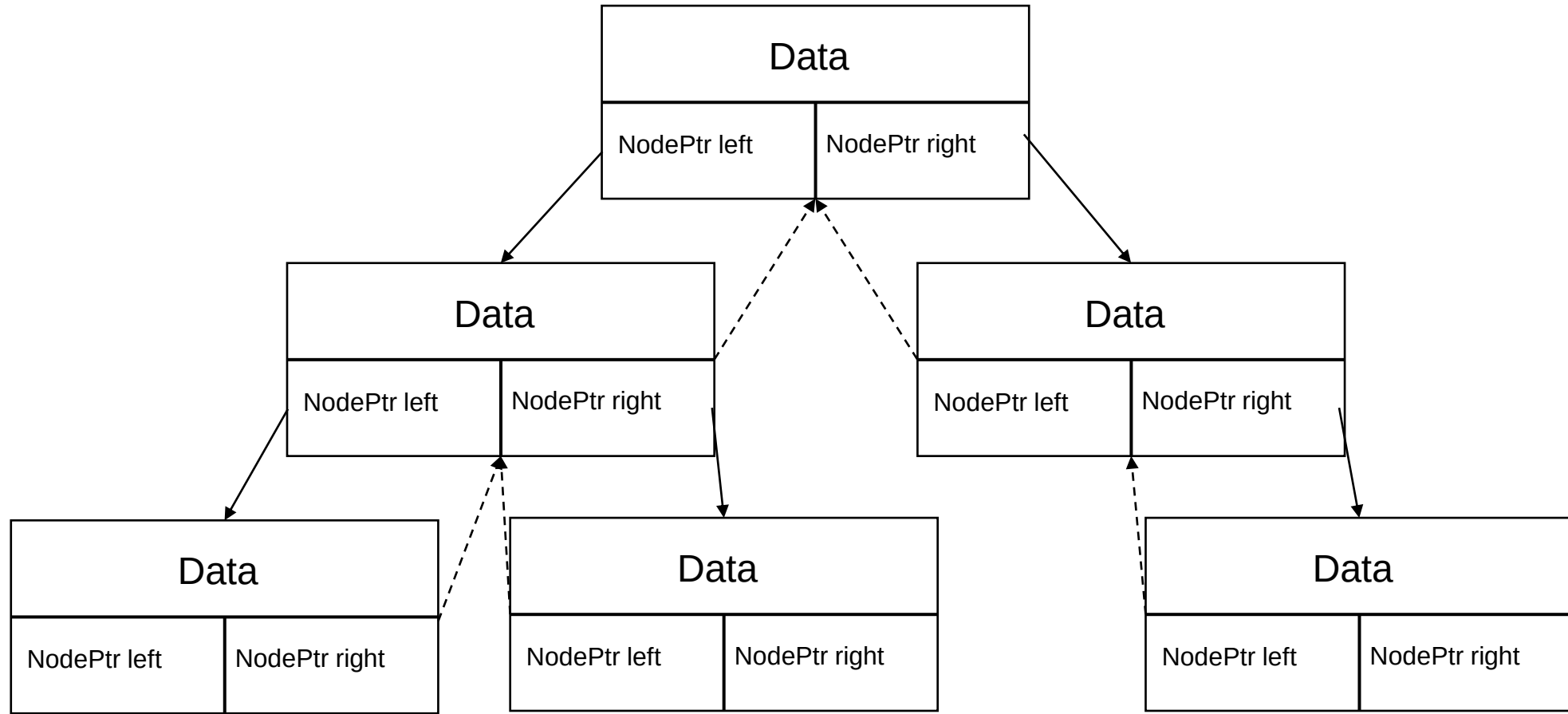
Счетчик ссылок

- Как хранить счетчик ссылок?
- Точно не статической переменной класса и не полем `size_t`
- Можно хранить указатель на выделенный на куче счетчик
- Но как реализовать `make_shared`?
- Нужен `ControlBlock`, который будет хранить счетчик и сам объект
- Как различать ситуацию между наличие и отсутствием `ControlBlock`?
- Можно хранить `bool` или использовать биты указателя

Дерево на умных указателях



Дерево на умных указателях со ссылкой на родителя



weak_ptr

- Получились циклические зависимости
- Память никогда не освободится, потому что счетчик ссылок не станет равным нулю
- Можно использовать weak_ptr в качестве указателя на родителя
- Он не будет учитываться при подсчете ссылок при уничтожении объекта

weak_ptr - реализация

- weak_ptr должен уметь понимать, жив ли ещё объект
- Но сейчас мы освобождаем память после смерти объекта, тогда обращение к счетчику – UB
- Решение – освободить память только когда закончатся weak_ptr, а разрушать объект, когда закончатся shared_ptr
- Для этого в ControlBlock поддержим счётчик weak_ptr-ов
- Тогда можем сделать два ControlBlock – с двумя указателями и с объектом или без него

Deleter и Allocator

- Хотим сконструировать `shared_ptr` от кастомных делитера и аллокатора
- Но у `shared_ptr` только один шаблонный параметр – `T`
- Хотим уметь хранить разные типы и динамически их подменять
- Будем использовать `type erasure`

Type Erasure

- Иногда хотим стирать тип (например, тип делитера)
- Хотим уметь понимать, какой всё же тип был на самом деле
- На самом деле это уже умеет делать механизм виртуальных функций
- Воспользуемся таким трюком:
 1. Создадим базовый класс с виртуальными методами
 2. Создадим шаблонного наследника
 3. Определим необходимые виртуальные методы
 4. Будем создавать на куче объект наследника с нужным шаблонным параметром, но хранить указатель на родителя

Наследование

- При наследовании в `shared_ptr` все будет корректно и без виртуального деструктора – благодаря `ControlBlock` и `type erasure`
- Для `unique_ptr` это не сработает – у него `Deleter` зашит в шаблонный параметр. Нужно делать деструктор виртуальным.

Aliasing constructor

- Иногда хотим иметь `shared_ptr` на некоторую часть разделяемого объекта, например, на поле
- `shared_ptr( const shared_ptr<Y>& r, element_type* ptr );`
- При уничтожении объекта, если нет больше ссылок, вызывается `Deleter`
- При вызове `get` (а также разыменовании) возвращается `ptr`
- Обязанностью пользователя является следить, чтобы время жизнь объекта под `ptr` совпадало с временем жизни разделяемого объекта

Касты

- Существуют касты `shared_ptr`:
 - `static_pointer_cast`
 - `dynamic_pointer_cast`
 - `const_pointer_cast`
 - `reinterpret_pointer_cast`
- Ведут себя как обычные касты
- Создаются новые `shared_ptr` с помощью `aliasing constructor` и преобразованного с помощью обычного каста указателя

Garbage Collectors

- **Tracing**

Обходим граф доступности объектов начиная. Все недостижимые объекты считаются мусором и удаляются

- **Reference Counting**

Каждый объект поддерживает счётчик ссылок на него, если счётчик равен нулю – объект удаляется

	Tracing	Reference Counting
Collection Style	Batch	Incremental
Cost Per Mutation	None	High
Throughput	High	Low
Pause Times	Long	Short
Real Time?	No	Yes
Collects Cycles?	Yes	No